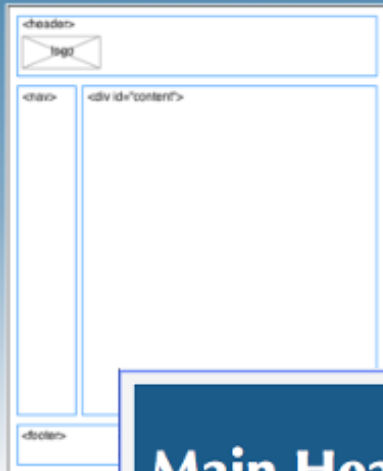


2 Column Layouts with CSS

Control the blocks on your web pages.



Main Heading Displaying on the Page

These will be
hyperlinks

This is a paragraph.

in the future

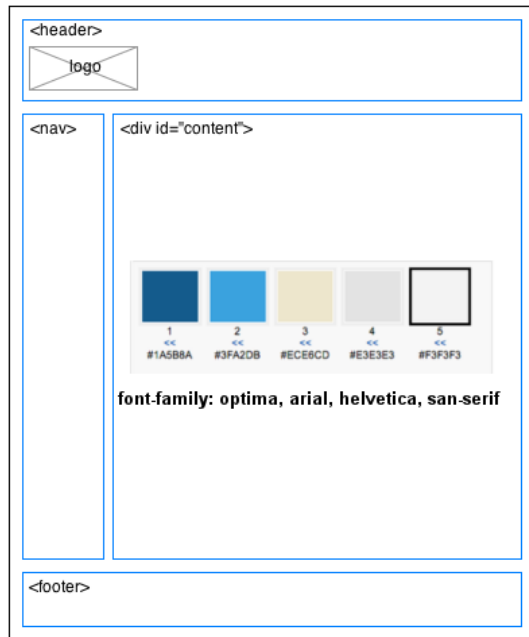
This is the footer information.

2 col and 3 col Layouts

You can use CSS to create fluid column layouts on a web page.

In this tutorial, you will learn how to create a 2-column layout. You will be able to use the end result as a template for future web pages.

Here is a sketch showing the layout we want to create as well as the color scheme we will use.



And here is the end result.



Basic HTML Page

Start with a basic HTML5 page, adding in elements to establish a structure that CSS can use to style the various blocks.

Here is the code. To really learn, TYPE IN THE CODE. You do not learn how to swim by watching people take swimming lessons. You have to get in the water and actually do it.

```

1 <!DOCTYPE html>
2 <html>
3 <!-- 2colLayoutDemo.html - template for pages with 2-column
   layout
4     Author:
5     Written:
6     Revised:
7 -->
8 <head>
9     <meta charset="utf-8" />
10    <title>2col Layout Page</title>
11    <style type="text/css">
12
13    </style>
14
15 </head>
16
17 <body>
18     <header>
19     <h1>Main Heading Displaying on the Page</h1>
20     </header>
21
22     <nav>
23     These will be hyperlinks<br /><br />
24     in the future<br />
25
26     </nav>
27
28     <div id="content">
29
30     <p>
31     This is a paragraph.
32     </p>
33     <!-- end of #content -->
34     </div>
35
36     <footer>
37     This is the footer information.
38     </footer>
39 </body>
40 </html>

```

Highlighted tags establish the blocks for CSS styling.

```

<!DOCTYPE html>
<html>
<!-- 2colLayoutDemo.html - template for pages with 2-column layout
    Author:
    Written:
    Revised:

```

```
-->
<head>
  <meta charset="utf-8" />
  <title>2col Layout Page</title>
  <style>

  </style>

</head>

<body>
  <header>
    <h1>Main Heading Displaying on the Page</h1>
  </header>

  <nav>
    These will be hyperlinks<br /><br />
    in the future<br />

  </nav>

  <div id="content">

    <p>
      This is a paragraph.
    </p>
    <!-- end of #content -->
  </div>

  <footer>
    This is the footer information.
  </footer>
</body>
</html>
```

If you view the page you should see something like this. None of the structural elements are apparent until we add in some CSS.

Main Heading Displaying on the Page

These will be hyperlinks

in the future

This is a paragraph.

This is the footer information.

Use CSS Border to See the Blocks

The CSS border is a great debugging tool that will allow us to see the HTML objects on the page.

Use a different color for each object. After the page is designed most of these border statements will be commented out.

Add this code to the <style> element:

```

8 <head>
9   <meta charset="utf-8" />
10  <title>2col Layout Page</title>
11  <style type="text/css">
12    header {
13      border:      1px solid purple;
14    }
15
16    nav {
17      border:      1px solid green;
18    }
19
20    footer {
21      border:      1px solid blue;
22    }
23
24    #content {
25      border:      1px solid black;
26    }
27
28  </style>
29
30 </head>
31

```

```

<style>
header {
border:      1px solid purple;
}

nav {

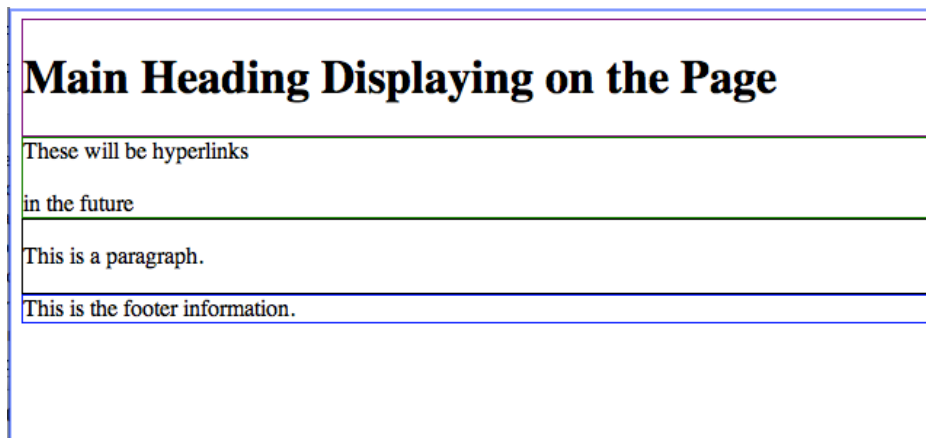
```

```
border:      1px solid green;
}

footer {
border:      1px solid blue;
}

#content {
border:      1px solid black;
}
</style>
```

Here is what the page should look like. Your colors may be different. That's okay.



Change width of <nav> block and float

Add a body selector the CSS to set the background color, establish a font-family, as well as a base font-size.

For the header selector, add in a margin for the top and bottom.

And finally, for the nav selector

1. Have it float left, allowing the other blocks to move up alongside the nav block
2. Set the margin on the right and bottom to 10px
3. Change the width to 20% of the page width.

Here is the code:

```

11 <style type="text/css">
12 body {
13   background:    #F4F4F4;
14   font-family:   optima, arial, helvetica, san-serif;
15   font-size:     18px;
16 }
17
18 header {
19   border:        1px solid purple;
20   margin:        10px 0px 10px 0px;
21 }
22
23 nav {
24   border:        1px solid green;
25   float:         left;
26   margin:        0px 10px 10px 0px;
27   width:         20%;
28 }
29

```

Here is the entire <style> element so far:

```

<style>
body {
background:    #F4F4F4;
font-family:   optima, arial, helvetica, san-serif;
font-size:     18px;
}

header {
border:        1px solid purple;

```



```
margin:    10px 0px 10px 0px;
}

nav {
border:    1px solid green;
float:     left;
margin:    0px 10px 10px 0px;
width:     20%;
}

footer {
border:    1px solid blue;
}

#content {
border:    1px solid black;
}
</style>
```

Here is what the page will look like. Don't worry that the content and footer overlap the nav area. We begin to fix those next.

Main Heading Displaying on the Page	
These will be hyperlinks	This is a paragraph.
in the future	This is the footer information.

Style the Content Block

Notice that most of the blocks use the standard HTML5 elements: `<header>` `<nav>` `<footer>`.

However, there isn't a specific element for content. There is `<article>` for things like blog postings, and `<section>` for groupings of things. But, none of these really fit the more general "content" that a page may have.

So, we built our own using the `<div>` element:

```
<div id="content">  
The main content of the page will go inside this element.  
</div>
```

The `id=""` attribute allows us to assign a unique name to this div. Whenever you see `id=""` you know there will be only one object on the page with that name. (The brother to `id=""` is `class=""` which allows blocks to all be named and styled the same.)

The value of `id=""` is up to you. Best practice: Keep it short and simple.

In the CSS have the `<div id="content">` float to the right and set the width to 80% of the page.

Notice that `#` is used to designate `id=""` in CSS. (Use the period `."` to designate a `class=""`).

```

23  nav {
24  border:      1px solid green;
25  float:      left;
26  margin:     0px 10px 10px 0px;
27  width:      20%;
28  }
29
30  footer {
31  border:      1px solid blue;
32  }
33
34  #content {
35  border:      1px solid black;
36  float:      right;
37  width:      80%;
38  }
39
40  </style>

```

Here is the entire <style> element so far:

```

<style>
body {
background:      #F4F4F4;
font-family:    optima, arial, helvetica, san-serif;
font-size:      18px;
}

header {
border:          1px solid purple;
margin:          10px 0px 10px 0px;
}

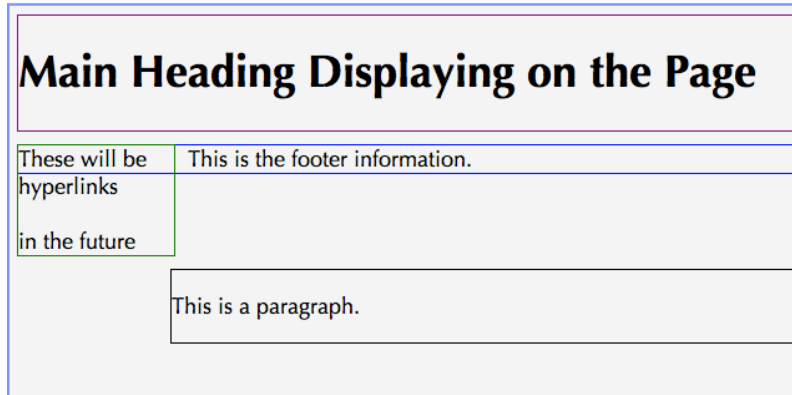
nav {
border:          1px solid green;
float:           left;
margin:          0px 10px 10px 0px;
width:           20%;
}

footer {
border:          1px solid blue;
}

#content {
border:          1px solid black;
float:           right;
width:           80%;
}
</style>

```

Here is what your page should look like. Don't worry about the footer, we will get to that next.



Ahh, but what happened?

With CSS, if a block cannot fit in the space it drops down. So, you have to remember to design a little loose so things can fit together.

Changing width to accommodate blocks

By simply changing the width of the #content from 80% down to 75% the browser automatically moves the content block up alongside the <nav> block. Yea!

Here is the code:

```

34  #content {
35  border:    1px solid black;
36  float:    right;
37  width:    75%;
38  }

```

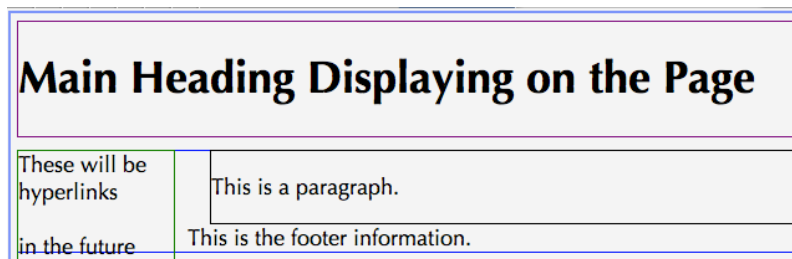
Here is the complete #content selector:

```

#content {
border:    1px solid black;
float:     right;
width:     75%;
}

```

And here is what the page should look like. Remember, we are going to remove most of the borders before we finish.



Next, we will deal with that pesky footer that wants to be on top of everything.

Use clear: both

float: properties remain in effect for the rest of the page. To stop a float you need to use the clear: property.

In the footer selector add a clear: both; This will stop both the left and right float that we set up for the nav and #content blocks.

Also, notice how we started fine-tuning the width for the nav and #content selectors.

```

23  nav {
24  border:      1px solid green;
25  float:      left;
26  margin:     0px 0px 10px 0px;
27  width:      22%;
28  }
29
30  footer {
31  border:      1px solid blue;
32  clear:      both;
33  }
34
35  #content {
36  border:      1px solid black;
37  float:      right;
38  width:      77%;
39  }
40

```

Here is the complete <style> element so far:

```

<style>
body {
background:    #F4F4F4;
font-family:   optima, arial, helvetica, san-serif;
font-size:     18px;
}

header {
border:        1px solid purple;
margin:        10px 0px 10px 0px;
}

nav {

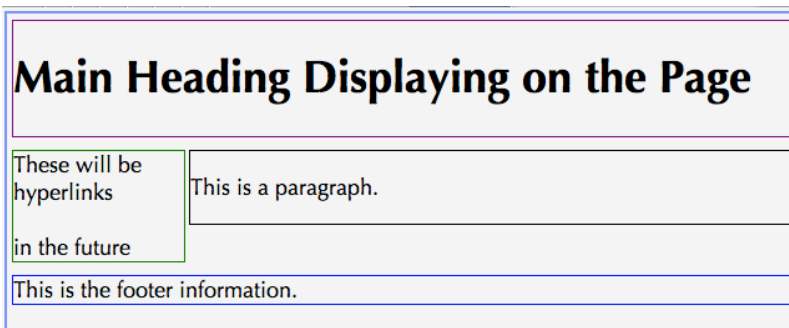
```

```
border:      1px solid green;
float:       left;
margin:      0px 0px 10px 0px;
width:       22%;
}

footer {
border:      1px solid blue;
clear:       both;
}

#content {
border:      1px solid black;
float:       right;
width:       77%;
}
</style>
```

Here is what the page looks like now that the floats are turned off by the footer selector.



Add padding to various blocks

By adding color and some padding to each object we can make the page easier to understand. Giving the nav and content blocks different colors tells the user that they have a different purpose.

We also changed the header to a dark blue and change the font to white. Although reverse-text is difficult (if not impossible to read) in long paragraphs, it can be especially effective with short, 3-5 word, phrases such as this page heading.

Here is the code. Notice that the colors are from the color scheme that we determined at the beginning of the project. It is often smart to include these hex codes and their color names in the comment block at the top of the HTML file.


```

11 <style type="text/css">
12 body {
13 background: #F4F4F4;
14 font-family: optima, arial, helvetica, san-serif;
15 font-size: 18px;
16 }
17
18 header {
19 background: #1A5B8A; /* dark blue */
20 border: 1px solid purple;
21 color: white;
22 margin: 10px 0px 10px 0px;
23 padding: 10px;
24 }
25
26 nav {
27 background: #ECE6CD; /* cream */
28 border: 1px solid green;
29 float: left;
30 margin: 0px 0px 10px 0px;
31 padding: 10px;
32 width: 22%;
33 }
34
35 footer {
36 border: 1px solid blue;
37 clear: both;
38 padding: 10px;
39 }
40
41 #content {
42 border: 1px solid black;
43 float: right;
44 padding: 10px;
45 width: 77%;
46 }
47
48 </style>

```

Here is the complete <style> element so far:

```

<style>
body {
background:      #F4F4F4;
font-family:    optima, arial, helvetica, san-serif;
font-size:      18px;
}

header {
border:         1px solid purple;
margin:         10px 0px 10px 0px;
}

nav {
border:         1px solid green;
float:          left;
margin:         0px 0px 10px 0px;
width:         22%;

```

```
}

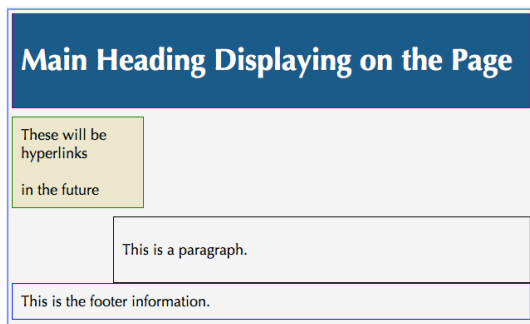
footer {
border:    1px solid blue;
clear:    both;
}

#content {
border:    1px solid black;
float:     right;
width:     77%;
}
</style>
```

Oops! What happened?

By changing the padding, margins, width, or border, the total width of an object can change. The #content block no longer fits the width of the page so the browser jumps it down below the nav block.

This is a common error that is easy to fix. Just adjust the padding, margins, borders, and widths of the blocks so everything will fit together.



Re-adjust margins, width, padding

Readjust the width for the nav and #content so everything fits.

```

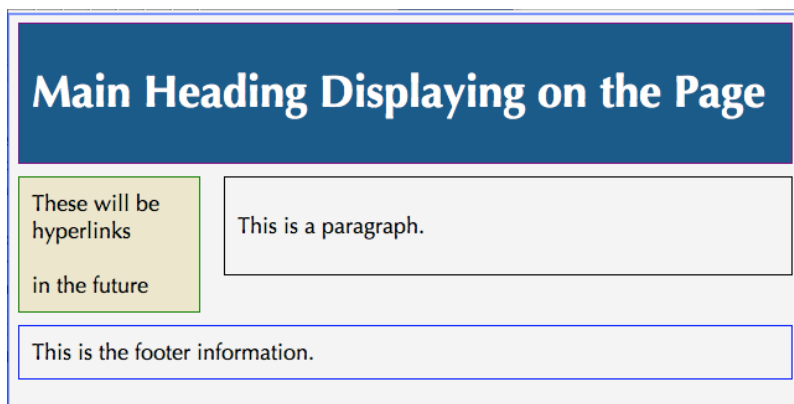
25
26 nav {
27   background: #ECE6CD;           /* cream */
28   border: 1px solid green;
29   float: left;
30   margin: 0px 0px 10px 0px;
31   padding: 10px;
32   width: 20%; ← Was 22%
33 }
34
35 footer {
36   border: 1px solid blue;
37   clear: both;
38   padding: 10px;
39 }
40
41 #content {
42   border: 1px solid black;
43   float: right;
44   padding: 10px;
45   width: 70%; ← Was 77%
46 }

```

For the nav selector change the width: 20%;

For the #content selector change the width: 70%;

Here is what the page should look like with these changes:



The Finished 2-Col Page

To finish the page, comment out the border: properties for the nav, footer, and #content.

Also, to make the page even more fluid:

- Remove the float: right; property. This will make the #content block automatically float: left
- Change the left margin so it will not overlap the nav block.

```

11 <style type="text/css">
12 body {
13   background: #F4F4F4;
14   font-family: optima, arial, helvetica, san-serif;
15   font-size: 18px;
16 }
17
18 header {
19   background: #1A5B8A;           /* dark blue */
20   border: 1px solid white;
21   color: white;
22   margin: 10px 0px 10px 0px;
23   padding: 10px;
24 }
25
26 nav {
27   background: #ECE6CD;           /* cream */
28   /* border: 1px solid green; */
29   float: left;
30   margin: 0px 0px 10px 0px;
31   padding: 10px;
32   width: 20%;
33 }
34
35 footer {
36   background: #1A5B8A;           /* dark blue */
37   /* border: 1px solid blue; */
38   clear: both;
39   color: white;
40   padding: 10px;
41 }
42
43 #content {
44   /* border: 1px solid black; */
45   margin: 0px 0px 10px 22%;
46   padding: 10px;
47   width: 75%;
48 }
49
50 </style>

```

comment out border:

Remove float:

Set left margin to accommodate width of nav block

Here is the complete <style> element:

(If you copy/paste this code you won't learn as much about CSS. Your brain will only be learning how to copy and paste.)

```

<style>
body {
background: #F4F4F4;
font-family: optima, arial, helvetica, san-serif;

```

```

font-size: 18px;
}

header {
background: #1A5B8A;          /* dark blue */
border: 1px solid purple;
color: white;
margin: 10px 0px 10px 0px;
padding: 10px;
}

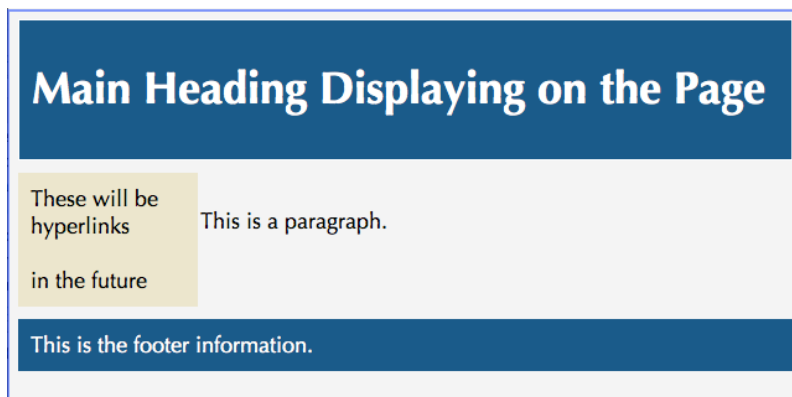
nav {
background: #ECE6CD;          /* cream */
border: 1px solid green;
float: left;
margin: 0px 0px 10px 0px;
padding: 10px;
width: 20%;
}

footer {
background: #1A5B8A;          /* dark blue */
/* border: 1px solid blue; */
clear: both;
color: white;
padding: 10px;
}

#content {
border: 1px solid black;
float: right;
padding: 10px;
width: 70%;
}
</style>

```

Here is what the finished 2-column layout will look like:



Summary

Be a smart programmer.

Save your finished 2-column file as 2colTemplate.html in a folder that is quick and easy to access. (You might want to put it where you have your basic template.html file stored.)

The next time you need a web page with a two-column layout, simply open up this file and use file/save as to save it to a new file name and in your current project folder.

What is the take away for this tutorial?

You now have a 2-col template to use in the future.

Here are some of the techniques that used:

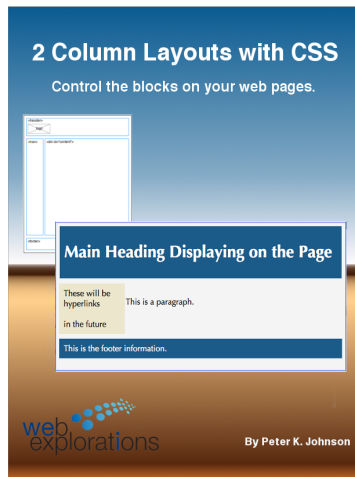
1. Sketching out a page layout, naming the blocks, and determining a color scheme before creating a web page.
2. Using the HTML5 elements to build a structure in the HTML code.
 - <header>
 - <nav>
 - <footer>
3. Using the <div> element along with the id=" " attribute to designate custom blocks inside the HTML.
4. Using the CSS float: property to allow blocks to "float" alongside each other.
5. Using the CSS border: property to debug and work with blocks
6. Understanding why blocks drop down even though they have the float: property.

Credits

CSS Layout, A tutorial written by [Peter K. Johnson - Web Explorations, LLC](http://WebExplorations.com) is licensed under a [Creative Commons Attribution-NonCommercial 3.0 Unported License](https://creativecommons.org/licenses/by-nc/3.0/).

Peter K. Johnson - <http://WebExplorations.com>

Updated 11-11-16



© 2013-16, Peter K. Johnson